🚗🔒🚨

ПРОЄКТ 4 · AUTOMOTIVE · ~7 ГОД · ESP-IDF v5.5.4

Бортовий блок керування: централізований замок, покажчики повороту та аварійна сигналізація

🎯 МЕТА ПРОЄКТУ

01 · Крок 0 · Обладнання, схема, безпека ⏱ 30 хв

Схема складання на макетній платі ⏱ додатково 15 хв

02 · Крок 1 · Ініціалізація проєкту ESP-IDF ⏱ 30 хв

03 · Крок 2 · Модульна архітектура: компоненти замість «одного файлу» ⏱ 30 хв

04 · Крок 3 · C для MCU: типи, `enum`, статична пам’ять ⏱ 25 хв

05 · Крок 4 · Драйвер навантаження: реле й зумер через BC547B ⏱ 40 хв

06 · Крок 5 · Кнопки: сире зчитування і дребезг на аналізаторі ⏱ 25 хв

07 · Крок 6 · Апаратне переривання: датчик удару ⏱ 45 хв

08 · Крок 7 · `esp_timer`: тік FSM (10 мс) і читання АЦП (50 мс) ⏱ 35 хв

09 · Крок 8 · Повна FSM-дебаунс кнопки (IDLE/PRESSED/HELD/RELEASE) ⏱ 50 хв

10 · Крок 9 · Логіка застосунку: замок, поворотники, аварійка ⏱ 40 хв

11 · Крок 10 · Потенціометр → швидкість блимання ⏱ 25 хв

12 · Крок 11 · Фінальна архітектура ⏱ 20 хв

13 · Крок 12 · Логи й діагностика ⏱ 15 хв

14 · Челенджі ⏱ ~1 год

Челендж 1 · П’ятий стан FSM — подвійний натиск (double-press) ⏱ 20 хв

Челендж 2 · Комбінація кнопок (LOCK + HAZARD одночасно) ⏱ 15 хв

Челендж 3 · Захист від “залипання” датчика удару ⏱ 15 хв

Челендж 4 · Калібрування дребезгу під конкретну кнопку ⏱ 10 хв

Челендж 5 · Лічильник подій у NVS ⏱ 20 хв

✅ ПІДСУМОК: ЩО МАЄТЕ ОТРИМАТИ В КІНЦІ

🚗🔒🚨

ПРОЄКТ 4 · AUTOMOTIVE · ~7 ГОД · ESP-IDF v5.5.4

Бортовий блок керування: централізований замок, покажчики повороту та аварійна сигналізація

> Покрокова практика: будуємо на ESP32-S3 (**чистий ESP-IDF, без Arduino**) справжній «бортовий комп’ютер» кузова (Body Control Module, BCM) — той самий клас пристрою, що в реальному авто керує замками дверей, покажчиками повороту й аварійкою. Чотири кнопки працюють через **скінченну машину станів (FSM)** із Модуля 2.6 — кожна відрізняє короткий і довгий натиск. «Датчик удару» (аварійна кнопка) підключений як **справжнє апаратне переривання** (Модулі 2.3–2.4), а не через опитування — це принципова різниця, яку ви побачите на логічному аналізаторі. Реле та зумер вмикаються через транзистор BC547B (Модуль 2.2) — жодне навантаження не йде напряму на GPIO. Увесь час у системі — від дебаунсу до частоти блимання — контролюється апаратними таймерами `esp_timer` (Модуль 2.5). Код пишемо на C у стилі, придатному для мікроконтролера (Модуль 2.1): статична пам’ять, `enum` / `struct`, без `malloc`, без `String`.

🧩 ESP-IDF з нуля · 🏗️ Модульні компоненти · ⚡ Апаратні переривання · ⏱️ esp_timer · 🎛️ FSM-дебаунс · 🏆 Челенджі

Інтегрує Модулі 2.1 (C++ для MCU), 2.2 (пасивні/активні навантаження), 2.3–2.4 (апаратні переривання), 2.5 (таймери), 2.6 (FSM-дебаунс).

🎯 МЕТА ПРОЄКТУ

У кінці ви отримаєте:

1. Робочий **ESP-IDF проєкт** (`idf.py build / flash / monitor`) з правильною модульною архітектурою: окремі **компоненти** (`components/…`), а не один файл `main.c` на 1000 рядків.
2. Чотири кнопки, кожна — окремий екземпляр **FSM-дебаунсу** (IDLE → PRESSED → HELD → RELEASE), що розрізняє короткий (< 2 с) і довгий (≥ 2 с) натиск, і при цьому **жодного разу не пропускає** натискання, навіть коротше за 10 мс.
3. Одну кнопку («датчик удару»), що працює через **справжнє апаратне переривання GPIO** (`gpio_isr_handler_add`), а не через опитування — і чітке розуміння, **чому** для критичних до часу подій (crash-сенсор, airbag) переривання є обов'язковим, а не опцією.
4. Два незалежні періодичні `esp_timer` — один «тікає» FSM кожні 10 мс, другий раз на 50 мс читає потенціометр — наочний доказ, що в ESP-IDF можна мати кілька незалежних таймерів без жодного `delay()` , що блокує систему.
5. Драйвер силового навантаження на BC547B: реле (замок дверей / аварійний контур) і зумер (звукові сигнали) — з розрахунком базового резистора і поясненням, чому GPIO **ніколи** не можна підключати до котушки реле напряму.
6. Осцилограми з логічного аналізатора: «брудний» дребезг на вході кнопки й **чистий**, передбачуваний фронт реакції системи (реле/зумер) — наочний доказ, що дебаунс працює.
7. Повністю робочий сценарій: **замок дверей** (коротке = замкнути/відімкнути, довге = valet-режим), **покажчики повороту** (коротке = увімкнути/вимкнути, довге = 3-кратне миготіння перестроєння), **аварійка** (коротке = увімкнути/вимкнути, довге = діагностика усіх систем), **датчик удару** (миттєво, через переривання, вмикає аварійку й імпульс реле).

---

01 · Крок 0 · Обладнання, схема, безпека ⏱ 30 хв

Що на столі: ESP32-S3-DevKitC-1 N16R8 (USB-C, кабель **даних**), макетна плата, перемички, 2× транзистор **BC547B** (TO-92), резистори **330 Ом** та **220 Ом** (базові, по одному на транзистор), реле **Songle** (синій модуль), потенціометр **B10K**, 4× тактові кнопки, зумер, логічний аналізатор **24 МГц / 8 каналів**.

> **🔴 ДВІ ВАЖЛИВІ ОСОБЛИВОСТІ ЦЬОГО НАБОРУ — ПРОЧИТАЙТЕ ПЕРЕД ЗБИРАННЯМ**
>
> 1. **Немає flyback-діода (1N4001/1N4148).** Модуль 2.2 навчав, що котушка реле **обов'язково** потребує діод проти зворотної ЕРС. Якщо ваш релейний модуль Songle — це **готова плата** (5×5 см, зелена гвинтова клема + 3-піновий роз'єм VCC/GND/IN) — діод, резистор і транзистор вже впаяні на платі, і зовнішній BC547B для реле **не потрібен**: подаєте логічний сигнал прямо на `IN` . Якщо ж це **«голе» реле** (лише 5 виводів, без плати) — вам потрібен зовнішній BC547B, і діод стає **критично обов'язковим**. У цьому проєкті вважаємо, що маємо готовий **модуль** (це і є «синя Relay Songle»), тому керуємо напряму через `IN` , без зовнішнього транзистора на реле. Якщо у вас усе-таки «голе» реле — не перемикайте його часто, поки не докупите діод, і перечитайте Крок 4 нижче — там пояснено обидва варіанти.
> 2. **Немає викрутки для гвинтових клем реле.** Тому силову (вихідну) сторону модуля — контакти **COM / NO / NC** — у цьому проєкті **не підключаємо** до зовнішнього навантаження (це були б фари, клаксон, паливний насос у реальному авто). Ми використовуємо реле лише для демонстрації комутації: чути клацання, і на багатьох модулях є свій індикаторний LED. Керуючий 3-піновий роз'єм (`VCC` / `GND` / `IN`) — штирьовий, підключається звичайними дротами-перемичками, викрутка не потрібна.

```
ПІНАУТ ПРОЄКТУ (ESP32-S3, безпечні GPIO — уникаємо 0/3/19/20/26-32/39-46)

 GPIO4  ── вхід  (pull-down) ← BTN1  ЗАМОК ДВЕРЕЙ         (FSM, таймер-опитування)
 GPIO5  ── вхід  (pull-down) ← BTN2  ПОКАЖЧИК ЛІВИЙ        (FSM, таймер-опитування)
 GPIO6  ── вхід  (pull-down) ← BTN3  АВАРІЙКА              (FSM, таймер-опитування)
 GPIO7  ── вхід  (pull-down) ← BTN4  ДАТЧИК УДАРУ          (АПАРАТНЕ ПЕРЕРИВАННЯ!)
 GPIO10 ── вхід ADC1_CH9     ← потенціометр B10K (середній вивід) — швидкість блимання
 GPIO11 ── вихід             → база BC547B №1 (через 220 Ом) → зумер
 GPIO12 ── вихід             → IN релейного модуля Songle (напряму, без транзистора)

СХЕМА КНОПКИ (внутрішній pull-down ESP32-S3, код: GPIO_PULLDOWN_ONLY)
  3.3V ──[ кнопка ]──┬──► GPIOx
                     └── (за бажанням: 10 кОм ── GND, зовнішній дублюючий pull-down —
                          необов'язково, внутрішнього достатньо; корисно, якщо хочете
                          «чистіший» сигнал для логічного аналізатора)
   • Не натиснуто → LOW (0) ;  Натиснуто → HIGH (1)

СХЕМА ЗУМЕРА (активний, через транзисторний ключ — Модуль 2.2)
  3.3V (або 5V з USB) ──[ зумер + ]
                        [ зумер − ] ── колектор BC547B
  GPIO11 ──[ 220 Ом ]── база BC547B
  емітер BC547B ── GND (СПІЛЬНА земля з живленням зумера — обов'язково!)

  Розрахунок бази: Iб = (3.3В − 0.7В) / 220 Ом ≈ 11.8 мА — у межах допустимого
  струму GPIO ESP32-S3 (макс. ~40 мА, рекомендовано <20 мА при постійному навантаженні),
  а зумер споживає лічені мА — транзистор відкривається повністю (насичення).

СХЕМА РЕЛЕ (готовий модуль Songle — драйвер уже на платі)
  3.3V ──► VCC модуля
  GND  ──► GND модуля
  GPIO12 ──► IN модуля        (логіка модуля зазвичай LOW-active АБО HIGH-active —
```

```
                                перевірте написом на платі; у коді нижче — HIGH-active,
                                за потреби зміните на протилежну)
  COM/NO/NC ── НЕ ПІДКЛЮЧАЄМО (немає викрутки, дивись примітку вище)

СХЕМА ПОТЕНЦІОМЕТРА (дільник напруги — типове підключення B10K)
  3.3V ──[ вивід 1 ]
          [ повзунок ]──► GPIO10 (ADC1_CH9)
         [ вивід 3 ] ── GND
```

Схема складання на макетній платі ⏱ додатково 15 хв

> Нижче — фізичне розташування деталей на стандартній макетній платі (830 точок, дві силові рейки зверху й знизу, центральний розрив між стовпчиками `a-e` і `f-j`). ESP32-S3-DevKitC-1 фізично завеликий, щоб ставити його в цю саму макетну плату поруч із усіма деталями — тримайте його **окремо** й ведіть до плати лише перемичками (це і безпечніше, і залишає більше місця).

```
                ЛІВА СИЛОВА РЕЙКА (+ червона = 3.3V, − синя = GND)
  +  ──────────────────────────────────────────────────────────────────
  −  ──────────────────────────────────────────────────────────────────
       a  b  c  d  e     f  g  h  i  j
  1    .  .  .  .  .     .  .  .  .  .
  2    .  .  .  .  .     .  .  .  .  .   ← BTN1 (ЗАМОК): ніжки в a2/a4 та f2/f4
  3    .  .  .  .  .     .  .  .  .  .     (кнопка "верхи" через розрив плати)
  4    .  .  .  .  .     .  .  .  .  .   ← перемичка f2→GPIO4, перемичка a4→ЛІВА рейка (+)
  5    .  .  .  .  .     .  .  .  .  .
  6    .  .  .  .  .     .  .  .  .  .   ← BTN2 (ЛІВИЙ ПОВОРОТ): ніжки a6/a8, f6/f8
  7    .  .  .  .  .     .  .  .  .  .     перемичка f6→GPIO5, перемичка a8→ЛІВА рейка (+)
  8    .  .  .  .  .     .  .  .  .  .
  9    .  .  .  .  .     .  .  .  .  .   ← BTN3 (АВАРІЙКА): ніжки a10/a12, f10/f12
 10    .  .  .  .  .     .  .  .  .  .     перемичка f10→GPIO6, перемичка a12→ЛІВА рейка (+)
 11    .  .  .  .  .     .  .  .  .  .
 12    .  .  .  .  .     .  .  .  .  .   ← BTN4 (ДАТЧИК УДАРУ): ніжки a14/a16, f14/f16
 13    .  .  .  .  .     .  .  .  .  .     перемичка f14→GPIO7, перемичка a16→ЛІВА рейка (+)
 14    .  .  .  .  .     .  .  .  .  .
 15    .  .  .  .  .     .  .  .  .  .
 16    .  .  .  .  .     .  .  .  .  .
 17    .  .  .  .  .     .  .  .  .  .   ← BC547B №1 (buzzer): 3 ніжки в c18, c19, c20
 18    .  .  .  .  .     .  .  .  .  .     (пласка сторона транзистора — до себе; звірте
 19    .  .  .  .  .     .  .  .  .  .      розкладку E-B-C мультиметром/даташитом, вона
 20    .  .  .  .  .     .  .  .  .  .      відрізняється між виробниками!)
 21    .  .  .  .  .     .  .  .  .  .   ← резистор 220 Ом: між f19 (база) і g19,
 22    .  .  .  .  .     .  .  .  .  .     перемичка g19→GPIO11
 23    .  .  .  .  .     .  .  .  .  .   ← перемичка e18 (емітер, нижня ніжка)→ЛІВА рейка (−/GND)
 24    .  .  .  .  .     .  .  .  .  .   ← зумер "−" → e20 (колектор), зумер "+" → ЛІВА рейка (+)
 25    .  .  .  .  .     .  .  .  .  .
 26    .  .  .  .  .     .  .  .  .  .   ← потенціометр B10K: 3 ніжки в c26, c27, c28
 27    .  .  .  .  .     .  .  .  .  .     (крайні ніжки — живлення, середня — повзунок)
 28    .  .  .  .  .     .  .  .  .  .   ← перемичка e26→ЛІВА рейка (+), e28→ЛІВА рейка (−)
 29    .  .  .  .  .     .  .  .  .  .     перемичка f27→GPIO10 (повзунок/ADC)
       a  b  c  d  e     f  g  h  i  j
  +  ──────────────────────────────────────────────────────────────────
  −  ──────────────────────────────────────────────────────────────────

  Релейний модуль Songle НЕ встромляється в макетну плату (він на власній
  платі з 3-піновим роз'ємом) — кладіть його поруч і ведіть трьома окремими
  перемичками: VCC→ЛІВА рейка (+), GND→ЛІВА рейка (−), IN→GPIO12 (напряму з ESP32,
  не через макетну плату).
```

Порядок складання (робіть рівно в цій послідовності — так простіше перевіряти на кожному кроці):

1. Перемичкою з’єднайте `GND` ESP32-S3 із **синьою (−)** рейкою макетної плати.
2. Перемичкою з’єднайте `3V3` ESP32-S3 із **червоною (+)** рейкою макетної плати.
3. Мультиметром у режимі “прозвінка” переконайтесь, що між + і − рейками **немає** короткого замикання (опір — нескінченність/розрив), перш ніж підключати живлення.
4. Встановіть 4 кнопки, кожна — **впоперек центрального розриву** плати (це стандартний спосіб для 4-ногих тактових кнопок: розрив плати не дає їй “самозамкнутись”, поки не натиснеш).

5. Для кожної кнопки: одну ніжку (з боку `f`) з’єднайте перемичкою з відповідним GPIO (4/5/6/7), іншу ніжку (з боку `a`, через ряд нижче) — з **червоною (+)** рейкою.
6. Встановіть **один** BC547B (для зумера) так, щоб 3 ніжки потрапили в 3 **окремі** сусідні ряди тієї самої колонки (`c`/`d` чи `f`/`g`) — кожен ряд плати є окремим електричним вузлом лише в межах 5 точок (`a-e` чи `f-j` окремо), тому 3 ніжки транзистора **обов’язково** мають бути в 3 різних рядах.
7. Мультиметром (режим діода) підтвердіть, яка ніжка — база, яка — колектор, яка — емітер (порядок **відрізняється** між виробниками BC547B — не покладайтесь сліпо на “стандартну” розкладку).
8. Резистор 220 Ом — між базою транзистора і окремим вільним рядом; перемичкою з’єднайте цей вільний ряд з GPIO11.
9. Емітер транзистора — перемичкою до **синьої (−)** рейки (спільний GND!).
10. Зумер: “+” до **червоної (+)** рейки, “−” до ряду з колектором транзистора.
11. Потенціометр B10K: крайні ніжки — до `+` / `−` рейок (будь-яка з двох, полярність ролі не грає, лише інвертує напрямок зростання значення), середня ніжка (повзунок) — перемичкою до GPIO10.
12. Релейний модуль Songle (окремо від макетної плати, Крок 0 вище): `VCC` → `+`, `GND` → `−`, `IN` → GPIO12 напряму.
13. **Другий** BC547B і резистор 330 Ом — відкладіть окремо як запасний комплект (потрібен лише якщо ваше реле виявиться “голим”, без власної плати — Крок 0).
14. Ще раз мультиметром прозвоніть `+` / `−` рейки на коротке замикання — і лише тоді підключайте USB.

🔵 **ЩО СПОСТЕРІГАЄМО** Якщо на кроці 3 чи 14 мультиметр показав коротке замикання — НЕ підключайте USB. Найчастіша причина: перемичка випадково потрапила в сусідній стовпчик (`a` замість `b`) або транзистор вставлений не тими трьома ніжками в три різні ряди (наприклад, дві ніжки в одному ряду замкнули базу з колектором).

🔴 **ТЕХНІКА БЕЗПЕКИ** • Перемикання — лише при **вийнятому USB**. • Емітер BC547B і мінус зумера — обов’язково на **спільний GND** з ESP32 (інакше транзистор не має опорного рівня і не закриється чітко). • Не торкайтесь колектора BC547B одразу після вимкнення індуктивного навантаження (котушка реле, якщо коли-небудь підключите «голе» реле без діода) — можливий короткий високовольтний викид. • На АЦП-пін (GPIO10) — не більше 3.3 В (дільник потенціометра сам це забезпечує).

ПЛАН ПРАКТИКИ

Час	Крок	Що робимо
0:00–0:30	Крок 0	Схема, безпека, підготовка
0:30–1:00	Крок 1	Ініціалізація проєкту ESP-IDF (`idf.py`, `CMakeLists.txt`)
1:00–1:30	Крок 2	Модульна архітектура: компоненти замість одного файлу
1:30–1:55	Крок 3	C++/C для MCU: типи, `enum`, статична пам’ять (Модуль 2.1)
1:55–2:35	Крок 4	Драйвер навантаження: реле й зумер через BC547B (Модуль 2.2)
2:35–3:00	Крок 5	Кнопки: сире зчитування і дребезг на аналізаторі
3:00–3:45	Крок 6	Апаратне переривання: датчик удару (Модулі 2.3–2.4)
3:45–4:20	Крок 7	`esp_timer`: тік FSM (10 мс) і читання АЦП (50 мс) (Модуль 2.5)
4:20–5:10	Крок 8	Повна FSM-дебаунс кнопки (IDLE/PRESSED/HELD/RELEASE) (Модуль 2.6)
5:10–5:50	Крок 9	Логіка застосунку: замок, поворотники, аварійка
5:50–6:15	Крок 10	Потенціометр → швидкість блимання

Час	Крок	Що робимо
6:15–6:35	Крок 11	Збірка `app_main.c`, фінальна архітектура
6:35–6:50	Крок 12	Логи `ESP_LOGI` і діагностика
6:50–7:20	Челенджі	5 завдань на розширення

---

02 · Крок 1 · Ініціалізація проєкту ESP-IDF ⏱ 30 хв

> 🟢 **META** Створити коректний ESP-IDF-проєкт під ESP32-S3, зрозуміти, що робить кожен `CMakeLists.txt`, і отримати перший успішний `idf.py build`.

Термінал (ESP-IDF v5.5.4 з активованим `export.ps1` / `export.sh`):

```
idf.py create-project bcu_automotive
cd bcu_automotive
idf.py set-target esp32s3
```

Структура одразу після створення:

```
 bcu_automotive/
├── CMakeLists.txt             ← кореневий, "точка входу" збірки
├── main/
│   ├── CMakeLists.txt         ← реєструє main як компонент
│   └── bcu_automotive.c       ← перейменуємо на app_main.c
└── sdkconfig                  ← згенерується після build/menuconfig
```

Кореневий `CMakeLists.txt` (не чіпаємо, він завжди такий):

```
cmake_minimum_required(VERSION 3.16)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(bcu_automotive)
```

> 🟣 **РОЗБІР: чому саме так** - `include(project.cmake)` — це файл із самого ESP-IDF, що робить всю важку роботу: знаходить компоненти, налаштовує toolchain для Xtensa/RISC-V, підключає `sdkconfig`. Ви його ніколи не редагуєте — лише підключаєте. - `project(bcu_automotive)` — ім'я збірки; звідси беруться назви `.elf` / `.bin` файлів. - `idf.py set-target esp32s3` — обов'язковий крок для нового проєкту: він генерує `sdkconfig` під конкретний чіп (Xtensa LX7, PSRAM-налаштування N16R8 тощо). Без нього `idf.py build` збиратиме під дефолтний `esp32` (класичний), і флешинг на S3 не спрацює.

Перша збірка (перевірка інструментів):

```
idf.py build
idf.py -p COM5 flash monitor      # замініть COM5 на свій порт (Диспетчер пристроїв)
```

> 🔵 **ЩО СПОСТЕРІГАЄМО** Збірка створює `build/`. У моніторі (вихід із `idf.py monitor`, вихід — `Ctrl+]`) бачимо стандартний boot-лог ESP-IDF (`rst:0x1`, `boot: ESP-IDF v5.5.4...`). Це підтверджує, що тулчейн і плата налаштовані правильно — можна переходити до власного коду.

> 🟠 **МІНІ-ЗАВДАННЯ** Відкрийте `idf.py menuconfig` → `Serial flasher config` → перевірте виставлений `Flash size` (16 MB для N16R8) і `Component config → ESP PSRAM` → увімкнено Octal PSRAM. Це типова помилка новачків: забути виставити PSRAM/Flash під конкретний модуль

плати.

---

03 · Крок 2 · Модульна архітектура: компоненти замість «одного файлу» ⏱ 30 хв

🟢 **META** Розбити систему на незалежні **компоненти ESP-IDF** — так, як це роблять у реальних прошивках, а не тримати всю логіку в `app_main.c`. Кожен компонент має свій `CMakeLists.txt`, публічний `.h` (в `include/`) і приватну реалізацію `.c`.

Цільова структура проєкту:

```
bcu_automotive/
├── CMakeLists.txt
├── main/
│   ├── CMakeLists.txt
│   └── app_main.c              ← лише "склеює" компоненти, без бізнес-логіки
└── components/
    ├── button_fsm/              ← Модулі 2.3-2.4-2.6: кнопки, ISR, FSM-дебаунс
    │   ├── CMakeLists.txt
    │   ├── include/button_fsm.h
    │   └── button_fsm.c
    ├── load_driver/             ← Модуль 2.2: реле + зумер через транзистор
    │   ├── CMakeLists.txt
    │   ├── include/load_driver.h
    │   └── load_driver.c
    └── pot_sensor/              ← Модуль 2.5: АЦП потенціометра на таймері
        ├── CMakeLists.txt
        ├── include/pot_sensor.h
        └── pot_sensor.c
```

`main/CMakeLists.txt`:

```
idf_component_register(
    SRCS "app_main.c"
    INCLUDE_DIRS "."
    REQUIRES button_fsm load_driver pot_sensor
)
```

`components/button_fsm/CMakeLists.txt`:

```
idf_component_register(
    SRCS "button_fsm.c"
    INCLUDE_DIRS "include"
    REQUIRES driver esp_timer freertos esp_common
    PRIV_REQUIRES esp_driver_gpio
)
```

`components/load_driver/CMakeLists.txt`:

```
idf_component_register(
    SRCS "load_driver.c"
    INCLUDE_DIRS "include"
    REQUIRES driver
)
```

`components/pot_sensor/CMakeLists.txt`:

```
idf_component_register(
    SRCS "pot_sensor.c"
```

```
    INCLUDE_DIRS "include"
    REQUIRES esp_adc esp_timer
)
```

🟣 **РОЗБІР: чому саме так** - `idf_component_register()` — макрос ESP-IDF, що перетворює папку на компонент: компілює `SRCS`, робить заголовки з `INCLUDE_DIRS` видимими для інших компонентів, які на нього `REQUIRES`. - **`REQUIRES` vs `PRIV_REQUIRES`** — `REQUIRES` означає «мої публічні заголовки (`include/*.h`) використовують типи з цього компонента» (транзитивна залежність). `PRIV_REQUIRES` — «мені потрібно це лише всередині `.c`, зовнішній код про це не повинен знати». Правило: якщо тип з чужого компонента з'являється у вашому `.h` — `REQUIRES`; якщо лише в `.c` — `PRIV_REQUIRES`. Це та сама ідея **інкапсуляції**, що і `private` / `public` у класі. - **Чому три окремі компоненти, а не один `hardware/`?** Кожен відповідає за одну відповідальність (Single Responsibility): `button_fsm` нічого не знає про реле, `load_driver` нічого не знає про кнопки. Це дозволяє тестувати й переносити кожен компонент окремо — наприклад, `load_driver` буде **дослівно перевикористаний** у Проєкті 5 (DefenceTech) без жодної зміни коду. - **`main` — тільки «клей»** — `app_main.c` викликає ініціалізацію компонентів і більше нічого не робить сам. Це найважливіше архітектурне правило цього проєкту: бізнес-логіка ніколи не живе в `main`, лише у компонентах, якими він керує.

🔵 **ЩО СПОСТЕРІГАЄМО** Після створення порожніх файлів `idf.py build` поки що впаде (компоненти ще не мають реалізацій) — це нормально, продовжуємо в наступних кроках.

🟠 **МІНІ-ЗАВДАННЯ** Намалюйте на аркуші стрілками, хто на кого `REQUIRES`: `main` → всі три; чи повинен `button_fsm` знати про `load_driver`? (Відповідь у Кроці 9 — ні, їх пов'язує лише `main`.)

---

04 · Крок 3 · C для MCU: типи, `enum`, статична пам'ять ⏱ 25 хв

🟢 **МЕТА** Застосувати принципи Модуля 2.1 до заголовків, які писатимемо: без динамічної пам'яті, чіткі типи фіксованого розміру, `enum` замість «магічних чисел», `volatile` для змінних, які змінює переривання.

`components/button_fsm/include/button_fsm.h`:

```
#pragma once
#include <stdint.h>
#include <stdbool.h>
#include "driver/gpio.h"

#ifdef __cplusplus
extern "C" {
#endif

// Модуль 2.6: чотири стани FSM-дебаунсу — рівно як у теорії, жодних "магічних чисел"
typedef enum {
    BTN_STATE_IDLE = 0,
    BTN_STATE_PRESSED,
    BTN_STATE_HELD,
    BTN_STATE_RELEASE,
} button_state_t;

// Подія, яку FSM віддає застосунку РІВНО ОДИН РАЗ за натискання
typedef enum {
    BTN_EVENT_NONE = 0,
    BTN_EVENT_SHORT_PRESS,   // відпущено раніше, ніж LONG_PRESS_MS
    BTN_EVENT_LONG_PRESS,    // утримано довше LONG_PRESS_MS (подія одразу під час утримання)
} button_event_t;

// POD-структура (Plain Old Data) — без конструкторів, без вказівників на купу.
// Кожен екземпляр важить кілька байтів і живе у статичному масиві, ніякого malloc.
typedef struct {
    gpio_num_t   pin;            // на якому GPIO сидить кнопка
    button_state_t state;        // поточний стан FSM
```

```
    uint32_t     state_enter_ms; // мітка часу (millis) входу в поточний стан
    bool         long_fired;     // прапорець "довгий натиск уже віддано" (одноразовість)
} button_fsm_t;

// Константи часу — Модуль 2.6, exact values з теорії
#define BTN_DEBOUNCE_MS     50u     // стабільність перед PRESSED->HELD і HELD->RELEASE->IDLE
#define BTN_LONG_PRESS_MS   2000u   // поріг короткий/довгий натиск
#define BTN_FSM_TICK_MS     10u     // період опитування (esp_timer), Крок 7

void button_fsm_init(button_fsm_t *btn, gpio_num_t pin);
button_event_t button_fsm_tick(button_fsm_t *btn, uint32_t now_ms);

#ifdef __cplusplus
}
#endif
```

🟣 **РОЗБІР: чому саме так** - `#pragma once` **замість include guard руками** — коротше, і всі сучасні компілятори Xtensa/RISC-V у ESP-IDF його підтримують. - `extern "C"` — навіть якщо файл `.c`, цей блок робить заголовок безпечним для підключення з C++ (якщо колись перепишете компонент на `.cpp`) — символи не зазнають C++ name-mangling. - `stdint.h` **-типи (** `uint32_t` **) замість** `int` **/** `long` — на 32-бітному Xtensa різниця непринципова, але звичка писати **точний розмір** рятує на 8/16-бітних MCU і робить намір явним: “це мітка часу в мілісекундах, рівно 32 біти, ніколи не знак”. - `enum` **замість** `#define BTN_STATE_IDLE 0` — компілятор перевіряє типи в `switch`, а показ у GDB/логах показує ім’я стану, а не число. - **Структура** `button_fsm_t` **— POD, без прихованого стану** — це масив статичних об’єктів (Крок 8), розмір яких відомий на етапі компіляції. Жодного `new` / `malloc` — на мікроконтролері з обмеженим RAM фрагментація купи є реальним і небезпечним явищем при роботі 24/7 (Модуль 2.1). - `state_enter_ms` **— одне поле замість двох окремих таймерів** — ми зберігаємо лише момент входу в стан, а тривалість рахуємо як `now_ms - state_enter_ms` щоразу. Це економить пам’ять і усуває клас помилок “забули оновити другий таймер”.

🔵 **ЩО СПОСТЕРІГАЄМО** Поки що це лише заголовок — код не компілюється в бінарник, це нормально: спочатку проєктуємо контракт (що видно ззовні), потім — реалізацію (Крок 8).

🟠 **МІНІ-ЗАВДАННЯ** Порахуйте розмір `button_fsm_t` через `sizeof()` (додайте тимчасовий `printf` у `app_main`). Скільки байт “коштує” одна кнопка? Помножте на 4 — це весь бюджет RAM для кнопок цього проєкту.

---

05 · Крок 4 · Драйвер навантаження: реле й зумер через BC547B ⏱ 40 хв

🟢 **МЕТА** Реалізувати компонент `load_driver`, що безпечно вмикає зумер (через транзисторний ключ) і реле (напряму через модуль) — і зрозуміти межу відповідальності: GPIO керує **базою** транзистора, а не самим навантаженням (Модуль 2.2).

`components/load_driver/include/load_driver.h` :

```
#pragma once
#include "driver/gpio.h"

#ifdef __cplusplus
extern "C" {
#endif

#define LOAD_BUZZER_PIN   GPIO_NUM_11   // база BC547B через 220 Ом
#define LOAD_RELAY_PIN    GPIO_NUM_12   // IN релейного модуля (драйвер уже на платі)

void load_driver_init(void);
void load_relay_set(bool on);
void load_buzzer_set(bool on);
void load_buzzer_chirp(uint32_t ms);   // короткий "біп" (замок, підтвердження)

#ifdef __cplusplus
```

```
}
#endif
```

`components/load_driver/load_driver.c`:

```
#include "load_driver.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

void load_driver_init(void) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << LOAD_BUZZER_PIN) | (1ULL << LOAD_RELAY_PIN),
        .mode         = GPIO_MODE_OUTPUT,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);
    gpio_set_level(LOAD_BUZZER_PIN, 0);
    gpio_set_level(LOAD_RELAY_PIN, 0);
}

void load_relay_set(bool on) {
    gpio_set_level(LOAD_RELAY_PIN, on ? 1 : 0);   // HIGH-active модуль; інверсія — якщо ваш LOW-active
}

void load_buzzer_set(bool on) {
    gpio_set_level(LOAD_BUZZER_PIN, on ? 1 : 0);
}

void load_buzzer_chirp(uint32_t ms) {
    // Короткий блокуючий сигнал ДОПУСТИМИЙ лише для миттєвого "біп" (10-200 мс)
    // під час підтвердження дії користувача — НЕ використовуємо це у постійному циклі!
    load_buzzer_set(true);
    vTaskDelay(pdMS_TO_TICKS(ms));
    load_buzzer_set(false);
}
```

🟣 **РОЗБІР: чому саме так - `gpio_config_t` — це ESP-IDF-спосіб, а не `pinMode()`** — один виклик налаштовує одразу кілька пінів через бітову маску `pin_bit_mask` (`1ULL << pin`), а не по одному виклику на пін. `ULL` обов'язковий: без нього зсув для пінів >31 (на класичному ESP32) переповнив би 32-бітний `int` ; на S3 GPIO ≤ 48, але звичка правильна завжди. - **База через 220 Ом, а не напряму** — без резистора база транзистора для GPIO виглядає як майже коротке замикання (перехід база-емітер — це, по суті, діод), пін віддав би небезпечно великий струм. Резистор обмежує струм бази до безпечних ~12 мА, а транзистор все одно **повністю відкривається** (насичення) — керуючий струм крихітний, а комутований — на порядки більший. Це і є суть транзисторного ключа. - **Реле — напряму на `IN` , без зовнішнього транзистора** — тому що обраний нами модуль Songle **вже містить** свій транзистор, резистор і (за специфікацією виробника) flyback-діод на платі. Дублювати драйвер зовні — зайве і призводить до плутанини рівнів логіки. - **`load_buzzer_chirp()` навмисно блокуючий (`vTaskDelay`)** — і це **єдине** місце у всьому проєкті, де дозволена затримка, бо викликається вона з контексту звичайної FreeRTOS-задачі (не з ISR, не з таймера) як миттєва реакція на подію користувача. `vTaskDelay` , на відміну від `delay()` в Arduino, **не морозить процесор** — під час очікування планувальник FreeRTOS віддає час іншим задачам. Але викликати її з таймерного колбека чи, тим паче, з ISR — заборонено (Крок 6-7 пояснюють, чому). - **Спільна земля (Модуль 2.2)** — емітер BC547B і мінус зумера мають бути з'єднані з GND ESP32. Якщо зумер живиться від окремого джерела (не від плати) — це джерело теж повинно мати спільний GND, інакше транзистор не матиме опорного рівня напруги.

🔵 **ЩО СПОСТЕРІГАЄМО** Викликавши `load_buzzer_chirp(100)` з тимчасового тесту в `app_main` , чуємо короткий "біп". Канал логічного аналізатора на GPIO11 покаже чіткий прямокутний імпульс 100 мс.

🟠 **МІНІ-ЗАВДАННЯ** Виміряйте аналізатором затримку між командою і реальним клацанням реле (порівняно з зумером) — механічне реле завжди на кілька мілісекунд повільніше електроніки.

---

06 · Крок 5 · Кнопки: сире зчитування і дребезг на аналізаторі ⏱ 25 хв

> 🟢 **META** Побачити проблему “живцем”, перш ніж її вирішувати: підключити кнопку через внутрішній pull-down і подивитись на дребезг контактів на логічному аналізаторі — без цього кроку мотивація FSM-дебаунсу (Крок 8) залишається абстрактною.

Тимчасовий тестовий код (буде замінений у Кроці 8) — `main/app_main.c` :

```c
#include "driver/gpio.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

#define BTN1_PIN GPIO_NUM_4

static const char *TAG = "raw_read";

void app_main(void) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << BTN1_PIN),
        .mode         = GPIO_MODE_INPUT,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,   // внутрішня підтяжка ВНИЗ — natively в ESP32-S3
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,
    };
    gpio_config(&cfg);

    while (1) {
        ESP_LOGI(TAG, "BTN1 = %d", gpio_get_level(BTN1_PIN));
        vTaskDelay(pdMS_TO_TICKS(5));   // навмисно швидко — щоб зловити дребезг у логах
    }
}
```

> 🟣 **РОЗБІР: чому саме так** - `GPIO_PULLDOWN_ENABLE` **у** `gpio_config_t` — це ESP-IDF-еквівалент `INPUT_PULLDOWN` з Arduino: без нього “відпущений” пін висить у повітрі (floating) і ловить наведення. - `gpio_get_level()` **повертає** `0` **/** `1` **миттєво** — без жодної фільтрації. Саме тому під час фізичного натискання ви побачите в моніторі **пачку** швидких 0↔1 переходів, перш ніж рівень стабілізується на HIGH — це і є **дребезг контактів (bounce)**. - `vTaskDelay(5 тіків)` **навмисно короткий** — щоб не пропустити “вікно” дребезгу (зазвичай 1–20 мс); тут це діагностичний код, у фінальному проєкті так **ніколи** опитування не робимо (для цього є `esp_timer` , Крок 7).

> 🔵 **ЩО СПОСТЕРІГАЄМО** Підключіть канал 1 аналізатора до GPIO4. Один фізичний “клік” кнопки на екрані аналізатора — це не один чіткий фронт, а **пачка** з кількох переходів LOW↔HIGH за перші 1–20 мс. У Serial-моніторі те саме видно як швидку зміну `BTN1 = 0` → `1` → `0` → `1` за один дотик пальця.

> 🟠 **МІНІ-ЗАВДАННЯ** Виміряйте аналізатором реальну тривалість дребезгу вашої конкретної кнопки (зазвичай 2–15 мс). Це число — аргумент, чому поріг `BTN_DEBOUNCE_MS = 50` (Крок 8) із запасом.

---

07 · Крок 6 · Апаратне переривання: датчик удару ⏱ 45 хв

> 🟢 **META** Зрозуміти принципову різницю між **опитуванням** (Кроки 5, 8 — таймер раз на 10 мс перевіряє стан) і **апаратним перериванням** (цей крок — чіп сам будить процесор у момент зміни рівня на піні). Для датчика удару (краш-сенсор) затримка реакції на опитуванні (до 10 мс + FSM-дебаунс до 50-100 мс) неприйнятна — переривання реагує за **одиниці мікросекунд**.

Додаємо в `components/button_fsm/include/button_fsm.h` :

```
#define CRASH_SENSOR_PIN   GPIO_NUM_7

void crash_sensor_init(void (*on_crash_isr_deferred)(void));
```

У `components/button_fsm/button_fsm.c`:

```
#include "button_fsm.h"
#include "freertos/FreeRTOS.h"
#include "freertos/queue.h"
#include "esp_log.h"

static const char *TAG = "crash_isr";
static QueueHandle_t s_crash_queue = NULL;

// ISR-обробник — виконується в контексті переривання. ПРАВИЛА ISR (Модулі 2.3-2.4):
//  1) IRAM_ATTR обов'язковий — код мусить лежати в RAM, бо під час стирання/запису
//     SPI-флеша (наприклад, іншою задачею) читання коду З ФЛЕША заборонене, а
//     переривання може статись у будь-яку мить.
//  2) Ніякого ESP_LOGI, malloc, vTaskDelay — усе це не ISR-safe (можуть заблокувати
//     або чекати на ресурс, якого немає в контексті переривання).
//  3) Мінімум роботи: лише "розбудити" звичайну задачу через чергу/семафор.
static void IRAM_ATTR crash_isr_handler(void *arg) {
    uint32_t dummy = 1;
    BaseType_t woken = pdFALSE;
    xQueueSendFromISR(s_crash_queue, &dummy, &woken);
    if (woken) portYIELD_FROM_ISR();   // якщо розбуджена задача пріоритетніша — перемкнутись одразу
}

// Звичайна FreeRTOS-задача (НЕ ISR) — тут уже можна ESP_LOGI, buzzer, реле
static void crash_task(void *arg) {
    void (*callback)(void) = (void (*)(void))arg;
    uint32_t dummy;
    while (1) {
        if (xQueueReceive(s_crash_queue, &dummy, portMAX_DELAY)) {
            ESP_LOGW(TAG, "УДАР! Апаратне переривання GPIO%d", CRASH_SENSOR_PIN);
            callback();   // делегуємо реакцію застосунку (Крок 9), а не вирішуємо тут
        }
    }
}

void crash_sensor_init(void (*on_crash_isr_deferred)(void)) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << CRASH_SENSOR_PIN),
        .mode         = GPIO_MODE_INPUT,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .intr_type    = GPIO_INTR_POSEDGE,   // реагуємо на фронт НАРОСТАННЯ (LOW->HIGH)
    };
    gpio_config(&cfg);

    s_crash_queue = xQueueCreate(4, sizeof(uint32_t));
    gpio_install_isr_service(0);             // спільний ISR-сервіс для всіх GPIO переривань
    gpio_isr_handler_add(CRASH_SENSOR_PIN, crash_isr_handler, NULL);

    xTaskCreate(crash_task, "crash_task", 3072, (void *)on_crash_isr_deferred, 10, NULL);
}
```

🟣 **РОЗБІР: чому саме так** - `GPIO_INTR_POSEDGE` — переривання «озброюється» на конкретний тип фронту. `POSEDGE` = LOW→HIGH (наше pull-down: не натиснуто = LOW, натиснуто = HIGH, тому натискання — це якраз наростаючий фронт). Є ще `NEGEDGE`, `ANYEDGE`, `LOW_LEVEL`, `HIGH_LEVEL`. - `gpio_install_isr_service(0)` викликається **один раз** на всю прошивку — це спільний диспетчер переривань ESP-IDF, що дозволяє кільком пінам мати окремі `gpio_isr_handler_add()` без ручного написання спільного ISR на всі GPIO одразу. - **Чому ISR не робить нічого, крім `xQueueSendFromISR` ?** Переривання виконується з найвищим пріоритетом, перериваючи все інше, включно з ядром FreeRTOS. Якщо в ньому зробити щось повільне (лог, `printf`, доступ до флеша) — ви заблокуєте всю систему на непередбачуваний час або спричините crash (`Guru Meditation Error`). Правило Модулів 2.3-2.4: **ISR лише сигналізує, задача — обробляє**. - `xQueueSendFromISR` + `portYIELD_FROM_ISR` — стандартний ESP-IDF/FreeRTOS патерн “передачі естафети” з переривання у звичайну задачу. `woken == pdTRUE`

означає, що розбуджена задача (`crash_task`) має вищий пріоритет за ту, що виконувалась у момент переривання — тоді просимо планувальник перемкнутись негайно, а не чекати наступного тіка. - **`crash_task` із пріоритетом 10** — вище за типову задачу `app_main` (пріоритет 1), бо реакція на удар — найкритичніша подія в системі. - **Функція-колбек `on_crash_isr_deferred` передається ЗЗОВНІ** — компонент `button_fsm` не знає, ЩО саме робити при ударі (вмикати аварійку — це бізнес-логіка застосунку, Крок 9). Це той самий принцип інкапсуляції з Кроку 2: компонент кнопок не залежить від компонента навантажень.

🔵 **ЩО СПОСТЕРІГАЄМО** На аналізаторі: канал 1 — GPIO7 (кнопка удару), канал 2 — GPIO11 (зумер). Затримка між фронтом на каналі 1 і реакцією на каналі 2 — **одиниці мілісекунд**, і що найважливіше — вона **не залежить** від того, що в цей момент робить решта системи (навіть якщо `crash_task` "спав" в очікуванні черги, переривання будить його миттєво). Порівняйте це з реакцією звичайних кнопок (Крок 8) — там затримка завжди ≥ 50 мс (свідомий компроміс дебаунсу заради стабільності).

🟠 **МІНІ-ЗАВДАННЯ** Змініть `GPIO_INTR_POSEDGE` на `GPIO_INTR_ANYEDGE` і подивіться, скільки разів спрацює переривання на один дотик (через дребезг!). Поясніть, чому для критичного сенсора обирають **один конкретний фронт**, а не "будь-яку зміну".

---

08 · Крок 7 · `esp_timer` : тік FSM (10 мс) і читання АЦП (50 мс) ⏱ 35 хв

🟢 **META** Дати системі "пульс" через апаратний таймер, а не через `while(1) { …; vTaskDelay(); }` у одній задачі. Два незалежні періодичні `esp_timer` — наочний доказ Модуля 2.5: кожна підсистема має **власний**, точний і незалежний годинник.

`components/pot_sensor/include/pot_sensor.h` :

```h
#pragma once
#include <stdint.h>

#ifdef __cplusplus
extern "C" {
#endif

void pot_sensor_init(void);          // старт періодичного esp_timer на 50 мс
uint16_t pot_sensor_get_raw(void);   // останнє прочитане значення 0..4095 (кеш, без блокувань)

#ifdef __cplusplus
}
#endif
```

`components/pot_sensor/pot_sensor.c` :

```c
#include "pot_sensor.h"
#include "esp_timer.h"
#include "esp_adc/adc_oneshot.h"

#define POT_ADC_UNIT     ADC_UNIT_1
#define POT_ADC_CHANNEL  ADC_CHANNEL_9   // GPIO10 на ESP32-S3 = ADC1_CH9
#define POT_SAMPLE_MS    50u

static adc_oneshot_unit_handle_t s_adc_handle;
static volatile uint16_t s_last_raw = 0;   // volatile: пише таймер, читає app_main/pot_sensor_get_raw

static void pot_timer_cb(void *arg) {
    int raw = 0;
    adc_oneshot_read(s_adc_handle, POT_ADC_CHANNEL, &raw);
    s_last_raw = (uint16_t)raw;
}

void pot_sensor_init(void) {
    adc_oneshot_unit_init_cfg_t unit_cfg = { .unit_id = POT_ADC_UNIT };
    adc_oneshot_new_unit(&unit_cfg, &s_adc_handle);
```

```
    adc_oneshot_chan_cfg_t chan_cfg = {
        .bitwidth = ADC_BITWIDTH_DEFAULT,   // 12 біт на ESP32-S3 → 0..4095
        .atten    = ADC_ATTEN_DB_12,        // дозволяє міряти до ~3.3В (замість дефолтних ~1.1В)
    };
    adc_oneshot_config_channel(s_adc_handle, POT_ADC_CHANNEL, &chan_cfg);

    const esp_timer_create_args_t args = {
        .callback = &pot_timer_cb,
        .name     = "pot_sample",
    };
    esp_timer_handle_t timer;
    esp_timer_create(&args, &timer);
    esp_timer_start_periodic(timer, POT_SAMPLE_MS * 1000ULL);   // esp_timer рахує в МІКРОсекундах!
}

uint16_t pot_sensor_get_raw(void) {
    return s_last_raw;   // volatile uint16_t читається атомарно на 32-бітному ядрі — гонки даних немає
}
```

Таймер FSM-тіка — додаємо в `components/button_fsm/button_fsm.c`:

```
#include "esp_timer.h"

static void fsm_timer_cb(void *arg) {
    uint32_t now_ms = (uint32_t)(esp_timer_get_time() / 1000ULL);
    // Крок 8 покличе тут button_fsm_tick() для кожної з 3 кнопок-FSM (не для датчика удару!)
    app_fsm_tick_all(now_ms);   // "гачок" у застосунок, реалізація — Крок 9
}

void button_fsm_start_timer(void) {
    const esp_timer_create_args_t args = {
        .callback = &fsm_timer_cb,
        .name     = "fsm_tick",
    };
    esp_timer_handle_t timer;
    esp_timer_create(&args, &timer);
    esp_timer_start_periodic(timer, BTN_FSM_TICK_MS * 1000ULL);   // 10 мс = 10000 мкс
}
```

Додаємо в `components/button_fsm/include/button_fsm.h` (обов'язково — інакше `idf.py build` впаде):

```
void button_fsm_start_timer(void);

// Реалізується у main/app_main.c (Крок 9) — оголошуємо тут, бо викликає її button_fsm.c,
// а не навпаки (той самий "гачок", що й crash_sensor_init у Кроці 6).
void app_fsm_tick_all(uint32_t now_ms);
```

🔴 **ТИПОВА ПОМИЛКА КОМПІЛЯЦІЇ** Якщо пропустити цей заголовок, `idf.py build` впаде на `button_fsm.c` з `error: implicit declaration of function 'app_fsm_tick_all' [-Wimplicit-function-declaration]`. Сучасний toolchain ESP-IDF v5.5.4 (GCC) трактує виклик недекларованої функції як **помилку збірки**, а не попередження. Причина — сам патерн "гачка": `button_fsm.c` викликає функцію, яку **реалізує** `main.c` (Крок 9, на цьому етапі ще не написаний), тож компілятору потрібен лише **прототип**, щоб перевірити виклик, а не сама реалізація. Якщо після цього кроку `idf.py build` все одно лається на `app_fsm_tick_all` — перевірте, що заголовок справді підключений (`#include "button_fsm.h"`) в обох файлах, і що прототип має рівно той самий тип аргументу (`uint32_t`), що й визначення в Кроці 9.

🟣 **РОЗБІР: чому саме так - `esp_timer` рахує в мікросекундах** — `esp_timer_start_periodic(handle, period_us)`. Найпоширеніша помилка новачків в ESP-IDF: забути домножити мілісекунди на 1000. - **`esp_timer_get_time()`** повертає мікросекунди від старту системи (64-бітний лічильник, не переповнюється практично ніколи) — ділимо на 1000, щоб отримати "свій" `millis()`, як в Arduino, але явно і без прихованої магії. - **Callback `esp_timer` виконується в окремій системній задачі `esp_timer` (не в ISR!)** — на відміну від GPIO-переривання (Крок 6), тут **можна** викликати звичайні функції ESP-IDF (навіть `ESP_LOGI`, хоч і небажано занадто часто — забирає час у власного стеку таймерної задачі, спільного на всі `esp_timer`). Головне правило — тримати колбек **коротким і без довгих блокувань** (`vTaskDelay` тут теж небажаний), інакше він "з'їдає" час, призначений для колбеків інших таймерів. - **Два незалежні таймери, а не один "великий цикл"** — FSM кнопок тикає раз на 10 мс

(потрібна швидкість реакції для дебаунсу), АЦП потенціометра — раз на 50 мс (значення міняється повільно, часте опитування витрачає ресурс АЦП і CPU даремно). Кожна підсистема отримує **саме той** темп, який їй потрібен — це неможливо зробити чисто, якщо все живе в одному `while(1)` з одним `vTaskDelay()` . - **`volatile uint16_t s_last_raw`** — пишеться в контексті таймерної задачі, читається з іншої задачі (`app_main` / `button_fsm`). На 32-бітній архітектурі читання/запис `uint16_t` атомарні “самі по собі”, але `volatile` обов’язковий, щоб компілятор не закешував значення в регістрі й не “забув” перечитати його з пам’яті між ітераціями. - **`ADC_ATTEN_DB_12`** — атенюація (послаблення) вхідного сигналу перед АЦП; без неї опорна напруга ~1.1 В і потенціометр на 3.3 В “впреться” в максимум задовго до кінця повороту ручки. З `DB_12` діапазон вимірювання розширюється приблизно до 3.3В.

🔵 **ЩО СПОСТЕРІГАЄМО** Тимчасово додайте `ESP_LOGI("pot", "%d", pot_sensor_get_raw());` у `app_main` раз на секунду — крутіть потенціометр і спостерігайте плавну зміну 0…4095, незалежно від того, що в цей час роблять кнопки (два таймери працюють паралельно, не заважаючи один одному).

🟠 **МІНІ-ЗАВДАННЯ** Змініть `POT_SAMPLE_MS` на 500 і поясніть на власному прикладі (крутячи ручку швидко), чому для повільно змінюваної фізичної величини (положення ручки) рідше опитування — це не “гірше”, а **економія** CPU й енергії без втрати якості.

---

09 · Крок 8 · Повна FSM-дебаунс кнопки (IDLE/PRESSED/HELD/RELEASE) ⏱ 50 хв

🟢 **META** Реалізувати рівно ту FSM, що описана в теорії Модуля 2.6: чотири стани, дебаунс 50 мс на обох переходах (натискання і відпускання), поріг короткий/довгий натиск — 2000 мс, і жодного пропущеного натискання, навіть коротшого за 10 мс.

📎 **Примітка про рівні сигналу.** У теоретичному описі FSM (ваш конспект) кнопка малювалась через **pull-up** (натиснуто = LOW). У нашій схемі — внутрішній **pull-down** (натиснуто = HIGH, Крок 0/5). Щоб код нижче залишався зрозумілим незалежно від типу підтяжки, вводимо єдину функцію `raw_pressed()` , яка **абстрагує** це в одному місці — решта FSM працює лише з булевим “натиснуто / не натиснуто” і взагалі не знає, яка перед нею схема.

`components/button_fsm/button_fsm.c` (основна логіка):

```c
#include "button_fsm.h"

static inline bool raw_pressed(gpio_num_t pin) {
    return gpio_get_level(pin) == 1;   // pull-down: HIGH = натиснуто. Для pull-up — поміняти на == 0
}

void button_fsm_init(button_fsm_t *btn, gpio_num_t pin) {
    gpio_config_t cfg = {
        .pin_bit_mask = (1ULL << pin),
        .mode         = GPIO_MODE_INPUT,
        .pull_down_en = GPIO_PULLDOWN_ENABLE,
        .pull_up_en   = GPIO_PULLUP_DISABLE,
        .intr_type    = GPIO_INTR_DISABLE,   // важливо: НЕ переривання — це опитувана FSM!
    };
    gpio_config(&cfg);

    btn->pin            = pin;
    btn->state          = BTN_STATE_IDLE;
    btn->state_enter_ms = 0;
    btn->long_fired     = false;
}

// Викликається РІВНО РАЗ на 10 мс з esp_timer (Крок 7). Повертає подію АБО BTN_EVENT_NONE.
button_event_t button_fsm_tick(button_fsm_t *btn, uint32_t now_ms) {
    bool pressed = raw_pressed(btn->pin);
    button_event_t event = BTN_EVENT_NONE;

    switch (btn->state) {

    case BTN_STATE_IDLE:
```

```
        // Перехід "кнопка натиснена" → PRESSED (GPIO переходить у активний рівень)
        if (pressed) {
            btn->state          = BTN_STATE_PRESSED;
            btn->state_enter_ms = now_ms;
        }
        break;

    case BTN_STATE_PRESSED:
        // Ще не впевнені, чи це реальне натискання, чи дребезг.
        if (!pressed) {
            // Провал сигналу за час дребезгу — це і був дребезг, повертаємось у IDLE
            btn->state = BTN_STATE_IDLE;
        } else if (now_ms - btn->state_enter_ms >= BTN_DEBOUNCE_MS) {
            // Перехід "50 мс пройшло" → HELD (стабільно натиснено, дребезг вщух)
            btn->state          = BTN_STATE_HELD;
            btn->state_enter_ms = now_ms;   // тепер рахуємо тривалість УТРИМАННЯ від цієї миті
            btn->long_fired     = false;
        }
        break;

    case BTN_STATE_HELD: {
        uint32_t held_ms = now_ms - btn->state_enter_ms;

        if (!pressed) {
            // Перехід "кнопка відпущена" → RELEASE
            if (!btn->long_fired) {
                event = BTN_EVENT_SHORT_PRESS;   // відпустили РАНІШЕ довгого порогу → короткий натиск
            }
            btn->state          = BTN_STATE_RELEASE;
            btn->state_enter_ms = now_ms;
        } else if (!btn->long_fired && held_ms >= BTN_LONG_PRESS_MS) {
            // Довге утримання — подія віддається ОДРАЗУ, ще під час натискання
            // (не чекаючи відпускання) і РІВНО ОДИН РАЗ (прапорець long_fired)
            event = BTN_EVENT_LONG_PRESS;
            btn->long_fired = true;
        }
        break;
    }

    case BTN_STATE_RELEASE:
        // Дзеркально до PRESSED: чекаємо 50 мс стабільного LOW, щоб пройшов дребезг відпускання
        if (pressed) {
            // Дребезг "підняв" рівень назад — повертаємось у HELD, натискання триває
            btn->state = BTN_STATE_HELD;
        } else if (now_ms - btn->state_enter_ms >= BTN_DEBOUNCE_MS) {
            btn->state = BTN_STATE_IDLE;
        }
        break;
    }

    return event;
}
```

🟣 **РОЗБІР: чому саме так - `switch` по `enum`, а не ланцюжок `if/else if`** — компілятор попередить (з `-Wswitch`), якщо ви додасте новий стан у `enum`, але забудете обробити його тут — саме та “легкість розширення”, яку обіцяє FSM-підхід. - **Один `state_enter_ms` на всі стани** — під час `PRESSED→HELD` ми **перезаписуємо** мітку часу (`state_enter_ms = now_ms`), бо далі рахуємо вже інший інтервал — тривалість утримання, а не тривалість дебаунсу. Це економить пам’ять (не два поля, а одне) і виключає розсинхронізацію двох незалежних таймерів. - **`PRESSED`: провал сигналу → миттєво назад у `IDLE`, а не “чекати ще”** — якщо дребезг “відпустив” пін раніше 50 мс, це означає, що стабільного натискання ще не було; наступний фронт HIGH знову підніме нас у `PRESSED` і почне відлік 50 мс заново. Це і є механізм, що **гарантовано ігнорує дребезг**: лічильник дебаунсу ніколи не “накопичується” через перерваний сигнал. - **`long_fired` — прапорець одноразовості** — без нього подія `BTN_EVENT_LONG_PRESS` видавалася б **щоразу** на кожному тіку після 2000 мс (поки кнопку тримають), а не один раз. Скидається лише при вході в `HELD` (новий цикл натискання). - **Подія на короткий натиск видається у `HELD`, а не в `RELEASE`** — саме в момент `pressed == false` всередині `HELD` ми точно знаємо: (а) дребезг натискання вже пройшов (бо ми тут, у `HELD`), і (б) поріг довгого натиску **не** досягнуто (інакше `long_fired` був би `true`). Це рівно момент фізичного відпускання кнопки — логічно й очевидно для користувача. - **`RELEASE→HELD` при дребезгу відпускання** — дзеркальна логіка до `PRESSED`: якщо контакт “підстрибнув” назад під час відпускання, ми не втрачаємо стан утримання, а просто продовжуємо його. Кнопка жодного разу “не губиться” між станами, хоч би скільки тривав фізичний дребезг. - **10-мс гарантія (з теорії)** — оскільки `button_fsm_tick()` викликається кожні 10 мс апаратним `esp_timer`, максимальна

затримка виявлення фронту — 10 мс, і навіть натискання коротше за цей інтервал (майже неможливо фізично для людського пальця, але важливо для контактних датчиків) буде піймано на найближчому тіку, поки рівень ще стабільний.

Приклад використання (масив із 3 FSM-кнопок — датчик удару НЕ входить, він на ISR):

```
// у app_main.c або окремому "app_logic" — Крок 9
#define N_FSM_BUTTONS 3
static button_fsm_t s_buttons[N_FSM_BUTTONS];

void app_fsm_tick_all(uint32_t now_ms) {
    static const char *names[N_FSM_BUTTONS] = {"LOCK", "TURN_L", "HAZARD"};
    for (int i = 0; i < N_FSM_BUTTONS; i++) {
        button_event_t ev = button_fsm_tick(&s_buttons[i], now_ms);
        if (ev != BTN_EVENT_NONE) {
            // делегування в бізнес-логіку — Крок 9
        }
    }
}
```

🔵 **ЩО СПОСТЕРІГАЄМО** На аналізаторі: канал 1 — сирий сигнал GPIO кнопки (дребезг видно, як у Кроці 5), канал 2 — вихід зумера/реле, керований подією FSM. Незважаючи на дребезг на каналі 1, канал 2 перемикається **рівно один раз** на фізичне натискання — це і є доказ, що дебаунс працює.

🟠 **МІНІ-ЗАВДАННЯ** Тимчасово зменшіть `BTN_DEBOUNCE_MS` до 5 мс і подивіться, чи з'являються подвійні спрацювання на вашій конкретній кнопці (порівняйте з виміряним у Кроці 5 дребезгом).

---

10 · Крок 9 · Логіка застосунку: замок, поворотники, аварійка ⏱ 40 хв

🟢 **МЕТА** "Склеїти" три компоненти в осмислену бортову систему: кожна подія FSM і переривання удару перетворюється на конкретну дію над реле/зумером. Це єдине місце в проєкті, де компоненти `button_fsm` і `load_driver` "знайомляться" — і робить це саме `main`, а не самі компоненти (Крок 2).

`main/app_main.c` **(повний):**

```
#include "button_fsm.h"
#include "load_driver.h"
#include "pot_sensor.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_log.h"

static const char *TAG = "bcu";

#define N_FSM_BUTTONS 3
enum { IDX_LOCK = 0, IDX_TURN_L, IDX_HAZARD };
static button_fsm_t s_buttons[N_FSM_BUTTONS];

// --- Стан бортової системи (простий, без окремого enum-FSM - для цього проєкту достатньо булевих прапорців) ---
static bool s_locked      = false;
static bool s_valet_mode  = false;
static bool s_turn_l_on   = false;
static bool s_hazard_on   = false;

static void handle_lock_button(button_event_t ev) {
    if (ev == BTN_EVENT_SHORT_PRESS) {
        s_locked = !s_locked;
        load_relay_set(s_locked);            // імпульс реле = "клацання" замка
```

```
        load_buzzer_chirp(s_locked ? 80 : 40); // різний "біп" на замкнення/відмикання
        ESP_LOGI(TAG, "Замок дверей: %s", s_locked ? "ЗАМКНЕНО" : "ВІДІМКНЕНО");
    } else if (ev == BTN_EVENT_LONG_PRESS) {
        s_valet_mode = !s_valet_mode;
        load_buzzer_chirp(300);
        ESP_LOGW(TAG, "VALET-режим: %s", s_valet_mode ? "УВІМКНЕНО" : "ВИМКНЕНО");
    }
}

static void handle_turn_button(button_event_t ev) {
    if (ev == BTN_EVENT_SHORT_PRESS) {
        s_turn_l_on = !s_turn_l_on;          // toggle: сам блимає в окремій "фоновій" логіці (Крок 10)
        ESP_LOGI(TAG, "Лівий поворотник: %s", s_turn_l_on ? "УВІМКНЕНО" : "ВИМКНЕНО");
    } else if (ev == BTN_EVENT_LONG_PRESS) {
        ESP_LOGI(TAG, "Лівий поворотник: режим перестроєння (3 миготіння)");
        // Челендж 1 пропонує реалізувати рахунок рівно 3 миготінь
    }
}

static void handle_hazard_button(button_event_t ev) {
    if (ev == BTN_EVENT_SHORT_PRESS) {
        s_hazard_on = !s_hazard_on;
        ESP_LOGW(TAG, "АВАРІЙНА СИГНАЛІЗАЦІЯ: %s", s_hazard_on ? "УВІМКНЕНО" : "ВИМКНЕНО");
    } else if (ev == BTN_EVENT_LONG_PRESS) {
        ESP_LOGI(TAG, "Діагностика: перевірка реле й зумера…");
        load_relay_set(true);  load_buzzer_set(true);
        vTaskDelay(pdMS_TO_TICKS(200));
        load_relay_set(false); load_buzzer_set(false);
    }
}

// Викликається з esp_timer кожні 10 мс (Крок 7/8)
void app_fsm_tick_all(uint32_t now_ms) {
    button_event_t ev;

    ev = button_fsm_tick(&s_buttons[IDX_LOCK], now_ms);
    if (ev != BTN_EVENT_NONE) handle_lock_button(ev);

    ev = button_fsm_tick(&s_buttons[IDX_TURN_L], now_ms);
    if (ev != BTN_EVENT_NONE) handle_turn_button(ev);

    ev = button_fsm_tick(&s_buttons[IDX_HAZARD], now_ms);
    if (ev != BTN_EVENT_NONE) handle_hazard_button(ev);
}

// Реакція на апаратне переривання удару (Крок 6) — виконується у звичайній задачі crash_task,
// НЕ в ISR, тож тут дозволені ESP_LOGI, реле, зумер без обмежень.
static void on_crash_deferred(void) {
    s_hazard_on = true;                  // датчик удару ЗАВЖДИ вмикає аварійку (як у реальному авто)
    load_relay_set(true);
    load_buzzer_set(true);
    vTaskDelay(pdMS_TO_TICKS(150));
    load_relay_set(false);
    load_buzzer_set(false);
    ESP_LOGE(TAG, "!!! ДАТЧИК УДАРУ СПРАЦЮВАВ — аварійка увімкнена примусово !!!");
}

void app_main(void) {
    load_driver_init();
    button_fsm_init(&s_buttons[IDX_LOCK],   GPIO_NUM_4);
    button_fsm_init(&s_buttons[IDX_TURN_L], GPIO_NUM_5);
    button_fsm_init(&s_buttons[IDX_HAZARD], GPIO_NUM_6);

    crash_sensor_init(on_crash_deferred);   // Крок 6: справжнє апаратне переривання
    pot_sensor_init();                      // Крок 7: незалежний таймер АЦП
    button_fsm_start_timer();               // Крок 7: незалежний таймер FSM (10 мс)

    ESP_LOGI(TAG, "Бортовий блок керування запущено.");

    // app_main може завершитись природно — уся логіка вже живе в таймерах/задачах/ISR.
    // Залишаємо "фонову" задачу для блимання поворотників/аварійки (Крок 10).
    while (1) {
```

```
        vTaskDelay(pdMS_TO_TICKS(1000));
        ESP_LOGI(TAG, "стан: locked=%d valet=%d turnL=%d hazard=%d pot=%u",
                 s_locked, s_valet_mode, s_turn_l_on, s_hazard_on, pot_sensor_get_raw());
    }
}
```

🟣 **РОЗБІР: чому саме так** - `app_main` **не містить** `while(1) { читати_пін(); }` — уся реактивна логіка вже працює через `esp_timer` (Крок 7-8) і ISR-задачу (Крок 6). Залишковий `while(1)` у `app_main` — лише для періодичного статус-логу раз на секунду, він **нікому не заважає**, бо `vTaskDelay` віддає CPU іншим задачам. - `handle_*_button()` **— три окремі маленькі функції, а не один величезний** `switch` — кожна відповідає за одну кнопку, легко читається й тестується окремо (Модуль 2.1: маленькі функції з єдиною відповідальністю зменшують когнітивне навантаження і ризик помилки на мікроконтролері, де немає зручного дебагера “на кожному кроці”). - `on_crash_deferred` **викликає** `vTaskDelay` **— і це дозволено**, бо виконується вона у `crash_task` (звичайна FreeRTOS-задача, створена в Кроці 6), а не в самому ISR. Саме для цього й існував увесь механізм черги (`xQueueSendFromISR`) — щоб “перенести” обробку з обмеженого контексту переривання у контекст, де можна робити що завгодно. - **Датчик удару вмикає аварійку незалежно від** `s_hazard_on` **-логіки кнопки** — це свідоме архітектурне рішення: критична безпекова подія (удар) має **вищий пріоритет**, ніж стан, встановлений користувачем через звичайну кнопку.

🔵 **ЩО СПОСТЕРІГАЄМО** У моніторі раз на секунду бачимо рядок стану всієї системи. Коротке натискання LOCK — чуємо короткий “клац” реле і “біп”; довге (2+ с) — інший, довший сигнал (valet-режим). BTN4 (удар) реагує миттєво незалежно від того, що показує статус-лог.

🟠 **МІНІ-ЗАВДАННЯ** Додайте правило: якщо `s_valet_mode == true` , короткий натиск LOCK **ігнорується** (у реальних авто valet-режим блокує центральний замок для сервісменів). Де саме у `handle_lock_button()` ви поставите цю перевірку і чому саме там?

🔊 **ДОВІДКА: коли лунає звук, а коли ні** Так само, як і в Проєкті 5, не кожна кнопка/подія тут озвучена — свідомий вибір:

Кнопка / подія	Що робить	Звук
LOCK коротко	перемкнути замок	`load_buzzer_chirp(80)` замкнено / `chirp(40)` відмкнено
LOCK довго (≥2с)	перемкнути valet-режим	`load_buzzer_chirp(300)`
TURN_L коротко	перемкнути лівий поворотник	**тиша** — лише лог; звук блимання додає `blink_task` (Крок 10), і то лише поки увімкнено
TURN_L довго	режим перестроєння (заготовка)	**тиша** — лише лог, логіки миготінь тут ще немає (Челендж 1)
HAZARD коротко	перемкнути аварійку	**тиша** — лише лог; звук так само з’являється окремо через `blink_task`
HAZARD довго	діагностика реле/зумера	короткий сигнал ~200 мс (реле+зумер разом)
Датчик удару (переривання)	примусово вмикає аварійку	короткий сигнал ~150 мс, одноразово, у момент удару

Тобто якщо натискаєте TURN_L чи HAZARD і не чуєте нічого — це нормально: сам факт перемикання (`s_turn_l_on` / `s_hazard_on`) не супроводжується “біп”, звук зумера в цих двох кнопках з’являється лише опосередковано, через `blink_task` у Кроці 10 нижче, і **тільки поки** відповідний прапорець увімкнено.

---

11 · Крок 10 · Потенціометр → швидкість блимання ⏱ 25 хв

🟢 **META** Використати значення АЦП (Крок 7) для керування частотою блимання поворотників/ аварійки — так реальні автомобільні реле-переривники (flasher relay) мають злегка різний темп залежно від навантаження; у нас це керується вручну через ручку.

Додаємо “фонову” задачу блимання (окрему від esp_timer, бо тут потрібна змінна **частота, а не фіксований період):**

```
// у main/app_main.c, після app_main() або в окремому app_blink.c

static void blink_task(void *arg) {
    bool led_phase = false;
    while (1) {
        uint16_t raw = pot_sensor_get_raw();                // 0..4095 від Кроку 7
        uint32_t period_ms = 200 + ((uint32_t)raw * 600) / 4095;  // мапимо на 200..800 мс

        if (s_hazard_on || s_turn_l_on) {
            led_phase = !led_phase;
            load_buzzer_set(led_phase);   // імітуємо звук реле-переривника поворотника ("цок-цок")
        } else {
            load_buzzer_set(false);
            led_phase = false;
        }
        vTaskDelay(pdMS_TO_TICKS(period_ms / 2));   // половина періоду = один "півтакт" меандру
    }
}

// у app_main(), після button_fsm_start_timer():
xTaskCreate(blink_task, "blink_task", 2048, NULL, 5, NULL);
```

🟣 **РОЗБІР: чому саме так** - **Чому блимання — окрема FreeRTOS-задача, а не третій** `esp_timer` **?** Тому що період тут **сам змінюється** залежно від потенціометра щоразу заново — простіше і наочніше виразити це через `while(1) + vTaskDelay(period_ms/2)` у власній задачі, ніж постійно перестворювати `esp_timer` з новим періодом. Обидва підходи в ESP-IDF правильні; вибір залежить від того, наскільки часто змінюється період. - `period_ms = 200 + (raw * 600) / 4095` — лінійне відображення (map) діапазону АЦП 0..4095 на діапазон періоду 200..800 мс, той самий прийом, що й `map()` в Arduino, але виписаний вручну через ціле арифметичне ділення (без чисел з рухомою комою — Модуль 2.1: на MCU без FPU операції з `float` помітно повільніші). - **Один** `load_buzzer_set()` **і для поворотника, і для аварійки** — обидва в реальному авто видають той самий звук реле-переривника; окремого фізичного зумера на кожен канал у нас немає (обмежений набір деталей), тому логічно ділимо один спільний.

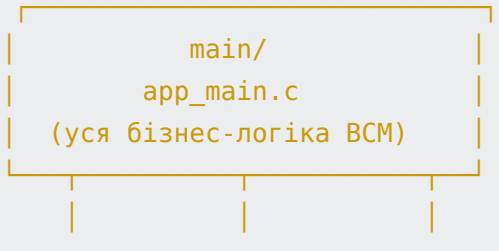
🔵 **ЩО СПОСТЕРІГАЄМО** Увімкніть аварійку (BTN3, коротко) і покрутіть потенціометр — темп клацання зумера плавно змінюється від швидкого (~200 мс/такт) до повільного (~800 мс/такт).

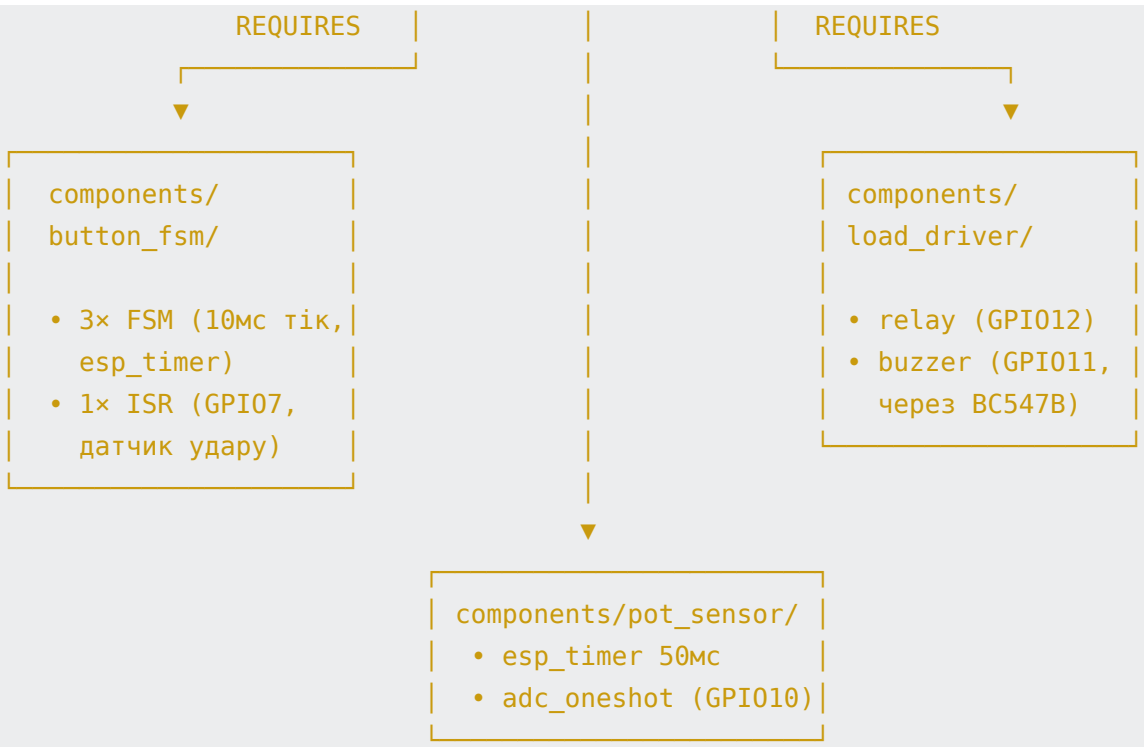
🟠 **МІНІ-ЗАВДАННЯ** Виміряйте аналізатором реальний період на каналі зумера при потенціометрі на мінімумі/максимумі — чи збігається з розрахунковими 200/800 мс?

---

12 · Крок 11 · Фінальна архітектура ⏱ 20 хв

🟢 **META** Побачити всю систему одним поглядом і переконатись, що жодна частина не “знає зайвого” про інші — це і є мета модульної архітектури з Кроку 2.





🟣 **РОЗБІР: чому саме так** - **Жодна стрілка не йде між `button_fsm` і `load_driver` напряму** — вони не `REQUIRES` одне одного (перевірте `CMakeLists.txt` , Крок 2). Це означає: можна винести `load_driver` в інший проєкт (наприклад, Проєкт 5) без жодної зміни, бо він нічого не знає про кнопки, і навпаки. - **Три незалежні “годинники” системи** — 10-мс таймер FSM, 50-мс таймер АЦП і апаратне переривання (без “годинника” взагалі, реагує миттєво) — працюють паралельно, кожен зі своїм темпом, і жоден не блокує інші.

🟠 **МІНІ-ЗАВДАННЯ** Домалюйте на цій схемі стрілку `blink_task` (Крок 10) — з чим вона взаємодіє? (Підказка: читає `pot_sensor_get_raw()` і викликає `load_buzzer_set()` — тобто це, по суті, ще один “клей”, як і `app_main` , тільки для одного вузького сценарію.)

---

13 · Крок 12 · Логи й діагностика ⏱ 15 хв

🟢 **МЕТА** Використати `ESP_LOGx` замість голого `printf` — стандартна практика ESP-IDF з рівнями важливості й тегами компонентів (Модуль 2.1: чистий, підтримуваний код).

```
ESP_LOGE(TAG, "..."); // Error   — щось критично не так (датчик удару)
ESP_LOGW(TAG, "..."); // Warning — важлива, але не аварійна подія (valet, аварійка)
ESP_LOGI(TAG, "..."); // Info    — звичайний робочий лог
ESP_LOGD(TAG, "..."); // Debug   — деталі для налагодження (вимкнено за замовчуванням)
```

🟣 **РОЗБІР: чому саме так** - **Рівень фільтрується глобально** через `idf.py menuconfig` → `Component config → Log output → Default log verbosity` — можна вимкнути весь `ESP_LOGD` / `ESP_LOGI` у “продакшн”-збірці однією зміною конфігурації, без редагування коду. - **`TAG` окремий на кожен `.c` -файл** (`static const char *TAG = "bcu";`) — у моніторі одразу видно, з якого компонента прийшов рядок, що критично при налагодженні системи з кількома паралельними задачами й таймерами.

🔵 **ЩО СПОСТЕРІГАЄМО** `idf.py monitor` кольорує рівні логів (червоний/жовтий/зелений) — миттєво видно критичні події (датчик удару) серед звичайного потоку статус-логів.

🟠 **МІНІ-ЗАВДАННЯ** Додайте `ESP_LOGD` з детальним станом FSM (`state` , `state_enter_ms`) у `button_fsm_tick()` і тимчасово увімкніть Debug-рівень у `menuconfig` , щоб “прожити” один цикл IDLE→PRESSED→HELD→RELEASE у логах.

---

14 · Челенджі ⏱ ~1 год

Челендж 1 · П’ятий стан FSM — подвійний натиск (double-press) ⏱ 20 хв

Додайте стан `BTN_STATE_WAIT_DOUBLE` між `RELEASE` і поверненням у `IDLE`: якщо протягом ~300 мс після відпускання стається нове натискання — це подвійний клік (наприклад, для поворотника — “3 миготіння перестроєння” з Кроку 9).

Челендж 2 · Комбінація кнопок (LOCK + HAZARD одночасно) ⏱ 15 хв

Реалізуйте виявлення “обидві кнопки утримуються одночасно ≥1с” як окрему умову поверх наявних FSM (перевірка стану обох `button_fsm_t` у `app_fsm_tick_all`) — класична автомобільна комбінація для входу в сервісне меню.

Челендж 3 · Захист від “залипання” датчика удару ⏱ 15 хв

Якщо `crash_isr_handler` спрацьовує частіше 5 разів за секунду (несправний контакт/наведення) — тимчасово вимкніть `gpio_isr_handler_remove()` на 2 секунди і залогуйте `ESP_LOGE`. Де саме порахувати частоту — в ISR чи в `crash_task`? Чому?

Челендж 4 · Калібрування дребезгу під конкретну кнопку ⏱ 10 хв

Виміряйте аналізатором реальний дребезг кожної з 4 кнопок окремо (він завжди трохи різний!) і зробіть `BTN_DEBOUNCE_MS` окремим полем у `button_fsm_t` замість глобальної константи — підберіть мінімальне безпечне значення для кожної кнопки.

Челендж 5 · Лічильник подій у NVS ⏱ 20 хв

Використайте `nvs_flash` (вбудований у ESP-IDF) для збереження лічильника “скільки разів замикали двері” між перезавантаженнями — перше знайомство з енергонезалежним сховищем на ESP32-S3.

✅ ПІДСУМОК: ЩО МАЄТЕ ОТРИМАТИ В КІНЦІ

- ☐ `idf.py build / flash / monitor` проходять без помилок під `esp32s3`.
- ☐ Модульна структура: 3 компоненти + `main`, кожен зі своїм `CMakeLists.txt`, без бізнес-логіки поза `main`.
- ☐ 3 кнопки коректно проходять повний цикл FSM IDLE→PRESSED→HELD→RELEASE→IDLE, розрізняючи короткий/довгий натиск, підтверджено на логічному аналізаторі (дребезг на вході vs один чіткий вихідний імпульс).
- ☐ 4-та кнопка (датчик удару) реагує через **справжнє апаратне GPIO-переривання** з передачею в задачу через чергу — і ви можете пояснити, чому не можна робити важку роботу прямо в ISR.
- ☐ Два незалежні `esp_timer` (10 мс FSM-тік, 50 мс АЦП) працюють одночасно, без взаємного блокування і без жодного `delay()`, що заморожує систему.
- ☐ Реле та зумер керуються безпечно (транзисторний ключ там, де потрібно; усвідомлення ролі flyback-діода навіть там, де його фізично немає).
- ☐ Потенціометр плавно керує темпом блимання поворотників/аварійки.
- ☐ Ви можете намалювати схему залежностей компонентів на аркуші й пояснити, чому саме так, а не одним файлом.